Learning What to Change: the poster, explained

Short speaker explainer for the visual-first review poster, 19 October 2026. REVIEW DRAFT for Toni, Chris and Pierce before anything is printed.

- **How to use it.** Read the plain text aloud. Grey bracketed lines are notes to you.
- **Numbers.** Every number is in `records/CORA_SUMMIT_CLAIM_LEDGER_20261017.md`, with its scope, and is audited against the poster in `POSTER_CLAIM_AUDIT.md`.
- **Figures.** Real figures are drawn from hash-pinned recorded data. The one illustrative figure says so.
- **The longer research dossier.** A deeper companion is kept separate; it is not available on Athe for this session. This explainer relies only on the records named here.

The 20-second version

I study how a reasoning system can tell what to change when its explanation of a problem is not good enough: the objects it sees, the rule it uses, or simply the evidence it has. My system, CORA, solves ARC puzzles by writing small programs. It once built a new rule on a real puzzle and checked it carefully. But a version that ignored its failure record did the same, and on 58 more puzzles construction added nothing. Learning which change helps is the open problem I am now testing.

The 90-second version

[full script, with pointing cues: POSTER_90_SECOND_SCRIPT.md]

- **The question.** When a reasoner's explanation is inadequate, can it work out whether it needs different objects, a different rule, or better evidence, and choose a change that improves predictions on new problems?
- **The test.** ARC gives two or three examples and a new input, so it is a clean place to test this.
- **Fitting is not explaining.** On 60 exposed training puzzles, CORA's checker accepted 6 answers, all correct. But on 26 puzzles every rule that fit the examples failed the test.
- **A rule CORA built.** On one real puzzle CORA built a new conditional rule: blue objects send a line up-left, red ones down-right. It rebuilt the rule three times with an example held out, and it was right.
- **The limit.** A version that ignored the failure record found the same rule, and on 58 further puzzles construction added zero verified solves.
- **Next.** CORA-v2 can already notice some inadequacy, such as competing hypotheses or an example the rule rests on alone. What is not yet shown is choosing the revision that helps.

The two-minute version

Add to the 90-second version:

- **Four reasons an explanation fails** (the illustrative panels): memorizing, wrong objects, a missing rule, too little evidence. A failing system does not know which one happened.
- **What the 0 of 58 suggests.** An evaluator-only check suggests most lie outside the single-rule language construction used: a hypothesis about expressivity, not proof of new semantics.
- **The neural baseline.** My reproduction of the third-party NVARC method scores 27.64; it is not CORA. Native CORA and CORA with rule construction score 0.00.
- **Biology.** What might transfer is the revision policy, not the grid operations: future work, untested.

1. The question: why objects, rules, evidence and revision are different problems

[poster banner]

The question, shaped in conversation with Toni Kazic: can a system induce useful representations and executable rules from a few observations, recognize when its explanation is inadequate, decide what must change, and carry that way of revising into a domain whose meaning is different? These are research aims. CORA has not achieved them all.

Words used on the poster, and here.

- **Representation**: how a grid is broken into objects and parts.
- **Rule**: executable instructions that change the grid.
- **Grammar** or **rule language**: the set of rules a system can express at all.
- **Memorization**: copying apparent regularities in the examples without a relation that transfers.
- **Evidence gap**: the examples do not distinguish between plausible hypotheses.
- **Leave-one-out**: rebuild the rule with one example hidden, and check that it predicts that example.
- **Failure diagnosis**: identifying an *observable* reason to doubt or revise the current explanation. It does not mean knowing that the hidden test answer is wrong; CORA never sees it.
- **Certified**: unique, and rebuilt in every leave-one-out check.
- **Exploratory**: executable and consistent with every example, but not certified. Never counted as a verified solve.
- **Generalization**: correct answers on new task instances, evaluated separately under a declared exposure boundary.

Why the four are different problems. The same failure, a wrong answer, can come from four places, and each needs a different repair:

- the *objects* were wrong: revise the representation;
- the *rule language* lacked the operation: extend the language;
- the examples did not *decide*: keep alternatives, abstain, or ask for evidence;
- the rule was *memorized*: a fit that does not transfer.

The wrong repair can break what already worked. For example, adding rules everywhere can turn a unique answer into a tie (page 7). The research problem is to tell these apart from the examples alone, choose a repair, and show that the choice beats doing everything blindly.

2. What an ARC task is (poster column 1)

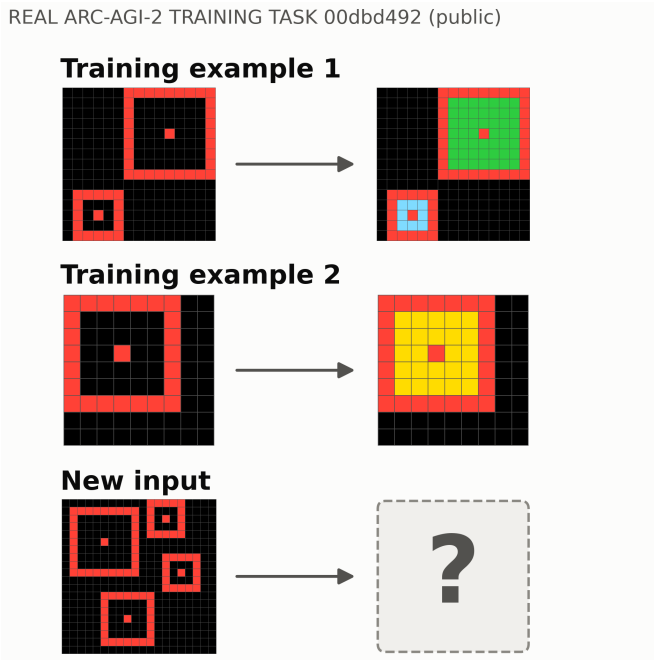
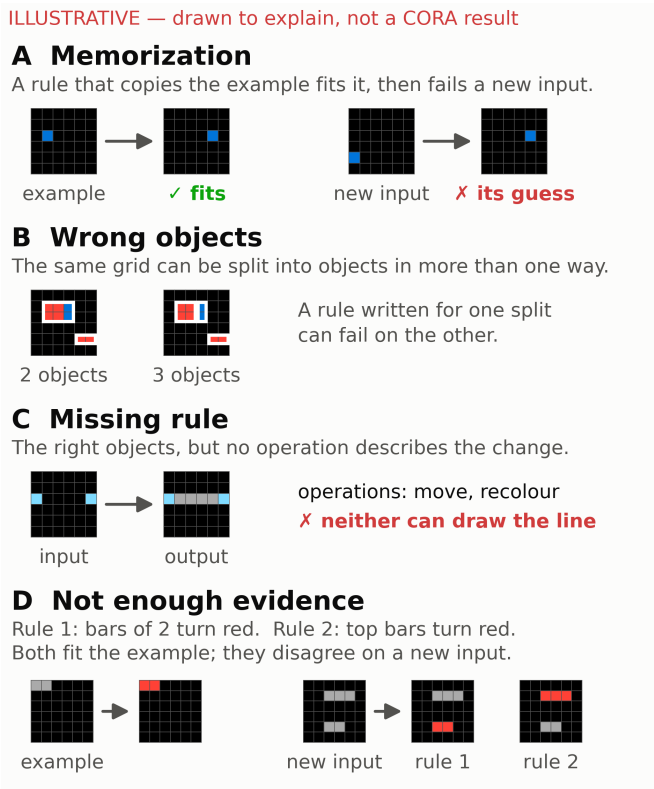


Figure: real task 00dbd492, a public ARC-AGI-2 training task. Grids are drawn cell for cell from the recorded bundle `v2_witness_00dbd492.json` (in `docs/summit_demo_20261023/data/`).

- **Semantics.** A grid is a small array of colours 0–9 (0 is black). A task gives a few training pairs (input → output) and a new input.
- **The task.** The solver must produce the new output exactly, with two attempts allowed. Here each frame is filled with a colour set by its size.
- **Memorization versus generalization.** A rule like "paint the cells that changed in example 1" fits example 1 and fails elsewhere: that is memorization. "Fill each frame with the colour its size calls for" generalizes.
- **Exposure.** This task was examined during CORA-v2 development; it is used here only to explain ARC.

The four illustrative panels

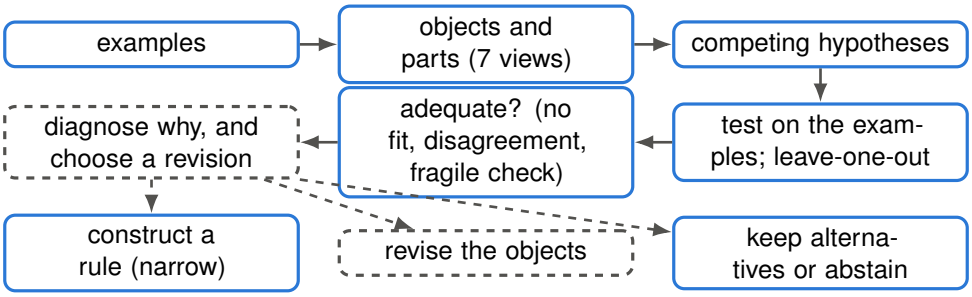


ILLUSTRATIVE: drawn to explain, not an ARC task and not a CORA result.

- **A.** A rule that copies the example's output fits it, then fails a new input.
- **B.** The same grid, split into 2 or 3 objects, depending on whether touching cells of different colours count as one object.
- **C.** The right objects, but the available operations (move, recolour) cannot draw the connecting line.
- **D.** "Bars of 2 turn red" and "top bars turn red" both fit the example and disagree on a new input.

3. CORA today: what is built, and what is a goal

[poster column 2 (pipeline) and column 3 (loop)]



Solid outlines: implemented and tested. Dashed: proposed, or not yet validated.

- **Native CORA (v23).** The frozen original system.
  - It segments a grid into objects in seven ways (“views”).
  - It searches for an object program that reproduces every example.
  - It accepts only answers that survive leave-one-out rebuilding.
  - Its whole harness adds other solver layers and gives two attempts.
- **CORA-FHR** (failure-conditioned hypothesis revision), the rule-construction experiment.
  - It reads native’s own failure trace and constructs a missing growth operation from a typed grammar of 37,750 expressions (G-grow).
  - It installs the operation reversibly and accepts it only if the unchanged engine uses it and every fold rebuilds it.
  - A **witness** is one recorded run that shows the whole chain end to end.
- **CORA-v2**, the hypothesis reasoner.
  - It keeps every hypothesis (representation, geometry, colour policy) that reproduces every example: its **version space**.
  - It abstains when they disagree, and reruns the search with each example held out.
  - It labels why it is unsure, with observable *diagnoses*, and offers exploratory guesses separately from certified answers.
- **Being built: a neural partner.** Candidate answers from the reproduced NVARC solver, checked by CORA’s executable hypotheses before they are combined (page 9). No result yet.
- **What is a goal, not a result.** Choosing the revision from the diagnosis (dashed). Revising the object representation (dashed). Transfer to science (page 13).

4. The real-task rule: ka95aa62f67d9 (poster column 2)

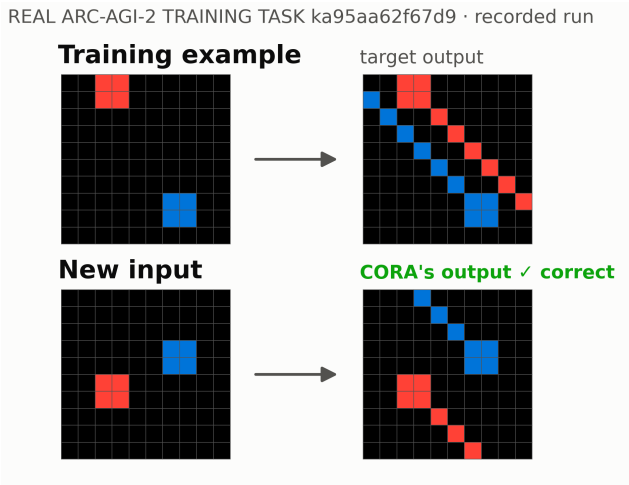


Figure. Real grids from the pinned bundle (`real_task.json` b06073a7, `real_witness.json` f0da3a8d): a training example with its target output, and the new input with CORA's output.

- **The rule CORA constructed.** If an object's colours are {1}, a ray goes north-west from its centre to the border, otherwise south-east. In ARC colours, **blue (1) objects send a line up-left; red (2) objects down-right**, each in the object's own colour.
- **The checks.** The rule was installed reversibly in the unchanged engine and rebuilt in 3 of 3 leave-one-out folds. The answer equals the evaluator's answer.
- **The comparison.** Frozen native CORA and matched extra search do not solve this task.

Why this is not hidden-set generalization.

- The failure autopsy chose this task, in-sample, and it is an exposed public training task.
- A failure-blind constructor, given the same evidence, found the same rule. So the run shows that CORA can *construct and verify* a missing rule. It does not show that diagnosing the failure *caused* the gain (a **causal** claim needs a controlled comparison that the diagnosis wins).
- It was developed after the CORA-FHR Kaggle entry and is not part of it.

The offline demo. `sh docs/summit_demo_20261023/run_demo.sh -real` needs Python with numpy. It is a *recorded replay*:

- it prints the recorded task, native's failure, the constructed rule, the engine check and the evaluator's verdict;
- it re-executes the constructed operation as standalone code (`guard_standalone.py`) on the test input's objects.

It is not a live rerun of the full engine. It was validated on 19 October (exit code 0).

5. What the experiments taught: the 60-task audit and the 58-task negative

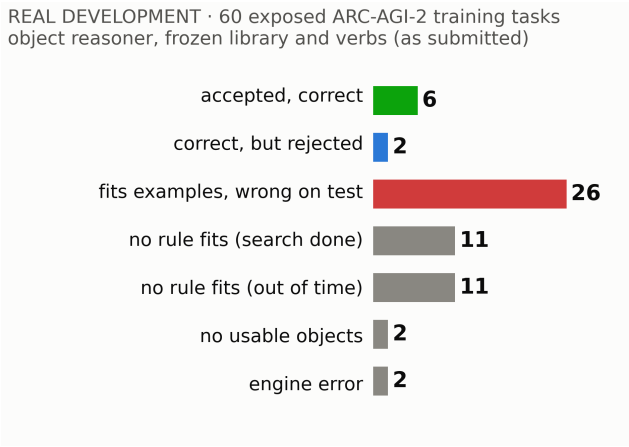


Figure: the 60-task audit. Real data (`eval_dev60_native_assets.json`, 54d8d744). It covers the native object reasoner with its frozen library and learned verbs, as submitted, on 60 exposed ARC-AGI-2 training tasks.

- Accepted and correct: 6. Accepted and wrong: 0.
- 26 tasks had rules that fit every example and were all wrong on the test: **memorization**.
- A correct rule existed but was rejected by the check on 2 tasks; no rule fit on 22.
- This is one branch. The whole harness got 12 of 60 over two attempts, and the two are never mixed.

The 58-task generalization test (negative)

- **Design.** 58 separately selected, still exposed training tasks; a protocol frozen before the run. Arms: frozen native, matched extra search, construction from the failure record, failure-blind construction, shuffled evidence, and a contrast arm.
- **Result.** No construction arm produced a certified answer (0 of 58), with 0 wrong and 0 lost. Frozen native solved 1. There were no discordant pairs, so the failure record showed no advantage, and the version was stopped.

Why so few constructions fire: a restricted diagnostic, not a proof

An evaluator-only (**oracle**) analysis, which reads the answers and is used only for diagnosis, asked one narrow question. Can a *single, unguarded, uniformly applied* G-grow expression explain the changed cells in any of native's seven views?

- For 50 of 58 tasks: no. In the sense defined here, they are **inexpressible** in that class.
- Of the other 8, 5 need colour behaviour no single such expression provides, and 3 pass the necessary checks.

This does *not* show that the tasks are impossible for CORA or need entirely new semantics. Guarded, deeper or re-segmented programs were not tested. It supports an expressivity hypothesis. Never say "56/58 impossible".

6. CORA-v2: many hypotheses, ambiguity and fragile checks (poster column 3)

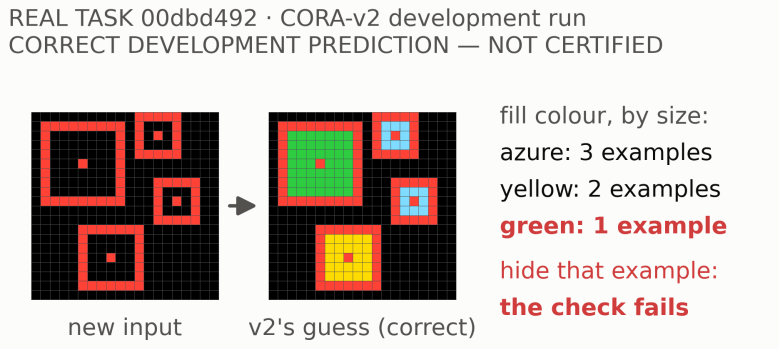


Figure: task 00dbd492, real data from the v2 engineering run.

- **The guess.** v2's 1,365 consistent hypotheses all agree, and the guess is correct. Frozen native and matched search have no certified answer.
- **Why it is not certified.** The test needs the colour for the size-32 frame (green), and only one example shows it. With that example hidden, the rebuilt rule has no entry for size 32 and abstains: it does not predict wrongly.
- **The diagnosis.** Checks on the examples rule out ambiguity, a wrong representation and a defect. The cause is *insufficient evidence*. A failure-blind reasoner gives the same answer.

It is an **exploratory**, uncertified prediction on an exposed task.

Engineering evidence on 118 exposed development tasks (in-sample)

	growth only (frozen v2)	growth + object selection
certified	0	0
58 pool tasks: exploratory correct / wrong	3 / 1	4 / 0
60 dev tasks: exploratory correct / wrong	1 / 0	2 / 0
mean CPU per task	20 s and 12 s	40 s and 27 s

- **More rules can mean less certainty.** On 22eb0ac0, adding object selection everywhere turned a unique answer into a disagreement. That is a concrete reason to revise where a diagnosis asks for it, not everywhere.
- **Where whole-object operations fall short.** On 22 development tasks objects change only in part:
  - native's built-in object parts fully explain 2 and partly explain 8;
  - 7 are unexplained, often by parts that touch other objects;
  - 5 show no partly changed object.

  A two-stage whole-object transformation produced 0 predictions on 32 routed tasks. These are planning results, not accuracy.
- **Two attempts** (evaluator-side replay, no submission). Adding v2's exploratory guesses to native's recorded attempts gives:
  - 58 pool tasks: 7 $\rightarrow$ 8 when filling only duplicate second attempts, 9 when always the second attempt;
  - 60 dev tasks: 12 $\rightarrow$ 12;
  - no answers lost.

  This is in-sample, so not evidence for an entry.

7. Inventing new operations: the automatic rule-language extension (Step B)

[poster column 3, “Inventing new operations”]

What it is. Step B is a separate CORA Level-4 experiment, in its own repository and run. It asks whether a system can construct genuinely new, reusable operations instead of depending only on operations a researcher specified.

- **K2.** It uses a frozen, generic, typed constructor language: a finite, closed set of type-directed semantic constructors, in 7 families of 19 schemas.
- **The run.** K2 is applied uniformly to failure clusters identified earlier, in a single run of 62 clusters.
- **What it must separate.** Repairing an existing learner versus inventing a new **semantic extension**: an operation whose meaning the old language could not express.

Status, 19 October 2026, from its counts-only administrative watcher.

- 475 of 497 K2 proposal units are done, with no errors.
- No completion marker and no output hash yet.
- No verdict on novelty or held-out transfer.

What still has to happen. When proposal generation finishes, the outputs are frozen and hashed before anyone inspects them. Then come a bounded novelty check and the reserved transfer evaluation. Finishing the proposals is not evidence of invention.

How it relates to CORA-FHR and CORA-v2. All three ask what to do when the current language is not enough.

- FHR constructs a missing operation from a fixed growth grammar for one task.
- v2 compares hypotheses and diagnoses why they are not enough.
- Step B tests whether a generic constructor language can produce new reusable operations.

Step B’s code is **not integrated** into CORA-v2 or any Kaggle entry, and none of its protected outputs were read for this poster.

8. What the neural baseline taught us, and what fusion would test

[poster bottom band, "Two approaches to ARC"]

What I reproduced. NVARC is a published neural ARC approach. I deployed it from the public NVARC notebook (originality audit, `records/KAGGLE_NEURAL_V1_ORIGINALITY_AUDIT_20261014.md`).

- **Unchanged:** the Qwen3-4B checkpoint, which the NVARC team pretrained on their synthetic data; their per-task LoRA test-time training; their decoding, candidate scoring and selection.
- **My six changes** were compatibility, coverage, diagnostics, provenance and robustness, not new algorithms.
- **Score.** It scored 27.64 on the official public board (Kaggle 57160429). It is a strong baseline, **not CORA**.
- **It does learn on each task:** its inference adapts the model to each puzzle's demonstrations. I did not invent or change that learning method.

For comparison, native CORA scored 0.00 (57144804), and CORA with rule construction 0.00 (57354496).

What the gap means, and what it does not. The tested neural system produces many more correct ARC predictions than the tested symbolic system. That does not show that neural networks reason and programs cannot. It does suggest asking what learned models could contribute *inside* CORA: representation proposals, program proposals, search guidance, uncertainty, and revision choice.

What combining them tested, and what it did not. A local audit filled NVARC's empty second attempts with CORA's answers on 240 exposed stand-in tasks. It gained nothing and lost one task: a correct 1×1 placeholder `[[0]]`. That tested *answer combination*. It did not test neural *proposals* feeding symbolic construction, diagnosis, revision and verification. That pipeline is untested.

The open question. Can learned neural representations and proposals help an executable reasoner find which part of a hypothesis needs revision?

The combination being tested (dashed on the poster: being built, not validated).



- **Verification.** A neural candidate grid, or a confident score, is a proposal, not a verification. CORA would accept a proposed rule only if it executes and reproduces every demonstration, and certify it only under the unchanged checks.
- **Fallback.** If no proposal helps, the output stays NVARC's unchanged.
- **Status, 19 October.**
  - The CORA side is built and unit-tested (`cora_neural_fusion/`): a typed proposal record; the check of which neural candidates CORA's executable hypotheses explain; fixed two-attempt policies with an exact no-op switch.
  - It has **not yet run on real NVARC outputs**, and there is no result. The saved NVARC run kept only its final answers, so an instrumented run that records the candidates and their scores comes first (`records/CORA_QWEN_FUSION_FEASIBILITY_20261019.md`).

A first experiment after the summit (a design idea, not authorized, not run). It would use exposed engineering tasks, with four conditions:

- CORA-v2;
- CORA-v2 with matched extra symbolic search;

- CORA-v2 with neural proposals (objects, parts, candidate rules);
- neural proposals chosen by observable diagnostic evidence.

Rules for the experiment:

- measure *candidate coverage* (is a correct hypothesis proposed?) separately from correct final predictions;
- neural outputs stay untrusted proposals until verified;
- if the neural model is adapted to a task's demonstrations, leave-one-out checks must repeat that adaptation without the held-out demonstration.

Modern LLMs can reason, use tools and check their work, and I do not claim otherwise. CORA asks a narrower question about inspectable, reversible, component-level revision.

9. Where this sits in prior work

The general loop of failing, revising and checking is not new. The possible difference is narrow and testable.

- **Popper** (Cropper and Morel, *Learning programs by learning from failures*, Machine Learning, 2021). Inductive logic programming that turns failed hypotheses into constraints that prune the search. It overlaps with using failure to guide search. CORA additionally asks which *component* (objects, rule language, evidence) to revise, in grid reasoning.
- **DreamCoder** (Ellis et al., PLDI 2021). Wake–sleep library learning that grows reusable abstractions and a neural search guide. It overlaps with growing a rule language, which is closest to Step B.
- **LILO** (Grand et al., ICLR 2024). LLM-guided program synthesis with library compression and auto-documentation. It overlaps with LLM-proposed programs and abstractions.
- **Model Discovery Agent** (Murphy, 2026). LLM-assisted Bayesian experiment design for data-efficient discovery of mechanistic models: recognizing model inadequacy, proposing new models, choosing experiments. It is closest to the scientific framing.

The narrow claim to be tested. A diagnosis-driven choice of *which component to revise* beats, under matched compute and matched evidence:

- a strong failure-blind choice;
- applying every revision;
- a shuffled diagnosis.

It must also add correct predictions on tasks the system was not developed on. None of these systems is said to be unable to do this.

10. How to test whether diagnosis helps

What a convincing test needs.

- **A fresh cohort.** Tasks not used in development, chosen by a rule declared beforehand.
- **Strong controls.**
  - frozen native;
  - matched extra search;
  - a strong failure-blind revision;
  - every revision applied everywhere;
  - the same at matched compute;
  - a shuffled diagnosis.
- **Harm counted.** Wrong certified answers and lost answers are counted, not only gains.
- **Separate tiers.** Certified and exploratory answers are reported separately.
- **A blind evaluator** that reads answers only after every prediction is frozen.
- **A pre-registered decision rule**, applied without retuning on the evaluated tasks.

A **prospective** test is one whose design and cohort are fixed before any result is seen.

Phase 2a status: RUNNING, RESULT PENDING.

- **Cohort.** 40 public training tasks.
  - They were selected by a declared rule and are independent of all v2 development.
  - A single authorized, metadata-only check excluded Step B's protected splits: 44 of 100 candidates were eligible, and the first 40 were used.
  - They are reused public tasks, so a result is a development evaluation, not a blind prospective test or an unseen-task score.
- **Arms.** Frozen native (A), matched extra search (B), and the frozen growth-only v2 (V).
- **GO rule.**
  - complete coverage and valid hashes;
  - zero wrong certified v2 answers;
  - at least 2 tasks where v2 is right and A and B are not.
- **Reporting.** The result will be added here and to the ledger only after its integrity checks.

If GO, Phase 2b. It would test the causal question: does the diagnosis choose better revisions than applying everything, at matched compute, or a shuffled diagnosis? It needs a new eligible cohort and approval.

If NO-GO. The version is closed. The next step would be one small, evidence-based change: part-aware or relational object parts, chosen from the 22-task characterization on page 7.

11. Science next, and how to reproduce everything

Plausible next domains. FUTURE, NOT TESTED: no biological transfer or mechanism is claimed. What might transfer is the diagnosis-and-revision policy. The operations, evidence and validators must be specific to each domain.

- **Protein conformational ensembles.** Is a structural hypothesis wrong, or is the ensemble representation too coarse?
- **Single-cell and other omics.** Is an effect a shift in average, a change in spread, or a change in the mixture of cell states? The level at which you infer (cells or donors) matters.
- **Maize lesion-mimic phenotyping.** Image geometry and topology tell you where and how lesions form, not which pathway is involved. Inferring a candidate pathway would need independent molecular or perturbation evidence.

Examples, data and wording are subject to Pierce's and Chris's input.

Reproducing what you see

- **Poster figures.** `python scripts/summit_visual_figures.py` writes `figures/*.png` and `POSTER_FIGURE_PROVENANCE.json`. It stops if any pinned input's sha256 differs.
- **Poster PDF.** From `docs/summit_20261023_reframe/`, run `SOURCE_DATE_EPOCH=1791763200 FORCE_SOURCE_DATE=1 pdflatex poster_visual_final_review.tex`.
- **Editable PowerPoint.** `python scripts/poster_visual_to_pptx.py`.
- **Demos** (offline; Python with numpy), both validated 19 October:
  - `sh docs/summit_demo_20261023/run_demo.sh` replays the synthetic witness;
  - adding `-real` replays the real ka95aa62f67d9 witness.

  Each replays the recorded run and re-executes the constructed operation standalone.
- **Records.**
  - ledger: `records/CORA_SUMMIT_CLAIM_LEDGER_20261017.md`;
  - v2: `records/CORA_REASONING_V2_*.md`;
  - rule construction: `records/CORA_FHR_*.md`.

Credits and disclosure. The research question was shaped in discussions with Toni Kazic. Authorship and credits are pending review by Toni, Chris and Pierce. AI coding assistance was used for parts of the implementation and documentation; the research, decisions and claims are the author's.